Advanced Data Science Project Report

Nexora: Time Series Forecasting and Customer Churn Prediction MVP

This report documents the design, implementation, evaluation, and deployment readiness of the Nexora machine learning MVP. The system combines a daily sales forecasting workflow with a customer churn prediction workflow inside one FastAPI application.

Prepared from the live repository artifacts, processed metrics, and project screenshots.

1. Executive Summary

Nexora is a production-oriented machine learning MVP built around two real datasets. The first pipeline converts transaction-level supermarket sales into a daily revenue time series and applies ARIMA to forecast future sales. The second pipeline uses a compact dense neural network to estimate the probability that a telecom-style customer will churn. The two models are exposed through a FastAPI backend, backed by automated preprocessing, saved model artifacts, a lightweight dashboard, and runtime monitoring.

The churn model achieved strong predictive performance with accuracy 0.96, F1-score 0.8462, and ROC-AUC 0.9939. The ARIMA forecaster was successfully trained and deployed, but its holdout performance (MAE 667.5171, MAPE 64.219%) shows that the daily revenue series is volatile and would benefit from richer features or a more expressive time-series model in a future iteration.

2. Business Problem and Objectives

The project addresses two operational questions. First, how can a retail team estimate near-term daily revenue to support inventory planning, staffing, and promotional decisions? Second, how can a customer success or retention team identify at-risk customers early enough to intervene? The goal of the MVP is not only to train machine learning models, but to package them inside a reproducible, testable, and deployable system suitable for demonstration or extension into production.

3. Dataset Overview

Dataset	Rows	Columns	Key Scope
supermarket_sales.csv	2000	14	Retail transactions from 2023-01-01 to 2023-12-31
customer_churn.csv	500	9	Binary churn labels with churn rate 10.6%

The sales dataset contains 6 product lines across branches A, B, C and cities Mandalay, Naypyitaw, Yangon. After aggregation, the raw data yields 364 observed daily points, and preprocessing expands this to 365 continuous calendar days. Total observed revenue in the source file is 519,280.69, with average daily revenue of 1,426.60.

The churn dataset contains 3 contract categories and 3 payment methods. The observed churn rate is 10.6%, which creates a class imbalance scenario. Paperless billing is enabled for 48.6% of customers, average tenure is 36.53 months, and average monthly charges are 113.64.

4. System Architecture

Layer	Implementation Detail
Data ingestion	CSV files loaded from the data directory using pandas
Preprocessing	Sales aggregation, missing-date filling, categorical encoding, and feature scaling
Model training	ARIMA for forecasting and Keras dense neural network for churn classification
Inference	Reusable predictor classes loading artifacts from data/processed
Serving	FastAPI routes for forecast, churn, health, and metrics
Frontend	Single-page HTML dashboard with fetch-based API calls
Monitoring	Request counts, latencies, errors, and endpoint usage persisted to JSON lines

Deployment	Dockerfile and docker-compose for containerized execution

The application follows a layered design that separates preprocessing, modeling, training, inference, serving, and monitoring. This makes the codebase easier to test and supports model loading once at application startup instead of per request, which reduces latency and improves operational stability.

5. Data Preprocessing Pipeline

5.1 Sales preprocessing

Sales preprocessing reads the supermarket transaction file, parses the Date column, aggregates Total revenue by day, and fills missing dates to produce a continuous daily time series. The pipeline also derives calendar signals such as day_of_week, month, and is_weekend. The resulting series is stored as daily_revenue.csv under data/processed so both training and inference can reuse the same consistent representation.

5.2 Churn preprocessing

Churn preprocessing removes the CustomerID identifier, ordinal-encodes contract type, one-hot encodes payment method, maps paperless billing to binary values, and standardizes numerical inputs such as tenure and charges. The dataset is then split with an 80/20 stratified split on the target to preserve churn balance. Scaler and encoder artifacts are persisted as pickle files to guarantee that training and inference use identical transformations.

6. Modeling Approach

6.1 Time series forecasting model

Attribute	Value
Model family	ARIMA
Selected order	(5, 0, 0)
Stationary on train split	True
ADF p-value	0.0
Training points	335
Test points	30
Training horizon end	2023-12-01
Evaluation horizon end	2023-12-31
MAE	667.5171
RMSE	813.1325
MAPE (%)	64.219

The forecasting pipeline uses an Augmented Dickey-Fuller stationarity check before fitting the ARIMA model. Based on the saved training artifact, the selected configuration was ARIMA(5, 0, 0). The final holdout metrics indicate that the baseline forecaster is functional and suitable for API demonstration, but not yet tuned for high-precision business forecasting. The report therefore treats the forecasting model as an honest baseline

and recommends future refinement rather than overstating its current business readiness.

6.2 Churn classification model

Attribute	Value
Model family	Dense neural network
Input feature count	9
Architecture	Dense(64, relu) -> Dropout(0.30) -> Dense(32, relu) -> Dropout(0.20) -> Dense(1, sigmoid)
Epochs run	40
Best validation loss	0.1514
Final validation AUC	0.9922
Accuracy	0.96
Precision	0.7333
Recall	1.0
F1-score	0.8462
ROC-AUC	0.9939

The churn model is intentionally compact because the dataset contains only 500 rows and an imbalanced positive class. Early stopping and class weighting are used to reduce overfitting risk and improve the model's sensitivity to churn cases. The resulting metrics are strong, especially the ROC-AUC of 0.9939, suggesting the model separates churn and non-churn customers well.

Confusion Matrix	Predicted No Churn	Predicted Churn
Actual No Churn	85	4
Actual Churn	0	11

7. API, UI, and Monitoring

Method	Endpoint	Purpose
GET	/	Serve the HTML dashboard
POST	/forecast/predict	Generate future revenue forecasts
GET	/forecast/history	Return recent observed revenue
GET	/forecast/model-info	Expose forecasting model metadata
POST	/churn/predict	Predict churn for one customer
POST	/churn/batch-predict	Predict churn for multiple customers
GET	/churn/model-info	Return neural network details and metrics
GET	/health	Report uptime and model load state

GET	/metrics	Return request counts, error rate, and latency

The FastAPI service acts as the integration point for both machine learning workflows. On startup, the application loads saved artifacts into memory and initializes a thread-safe metrics logger. A custom middleware records request latency, endpoint usage, and errors, then appends monitoring snapshots to monitoring/metrics_store.json. The dashboard consumes these endpoints through plain JavaScript fetch calls and provides a minimal but functional operator-facing interface.

8. Validation and Quality Assurance

The repository was validated through end-to-end execution of the training pipeline, unit testing, and API smoke checks. The consolidated training script successfully produced processed datasets, serialized model artifacts, and saved metrics files. The automated test suite passed 12 tests covering sales preprocessing, churn preprocessing, forecast inference, and churn inference behavior.

Validation item	Observed outcome
Training pipeline	Completed successfully and saved both model artifacts
Unit tests	12 tests passed
Health endpoint	Returned status ok with both models_loaded flags true
Forecast endpoint	Returned valid future dates and positive revenue values
Churn endpoint	Returned probability, binary prediction, and risk label

9. Screenshot Evidence

The following screenshots capture the implemented system at runtime. They document the live API, training output, health monitoring response, and browser dashboard used to demonstrate the MVP.

API Documentation

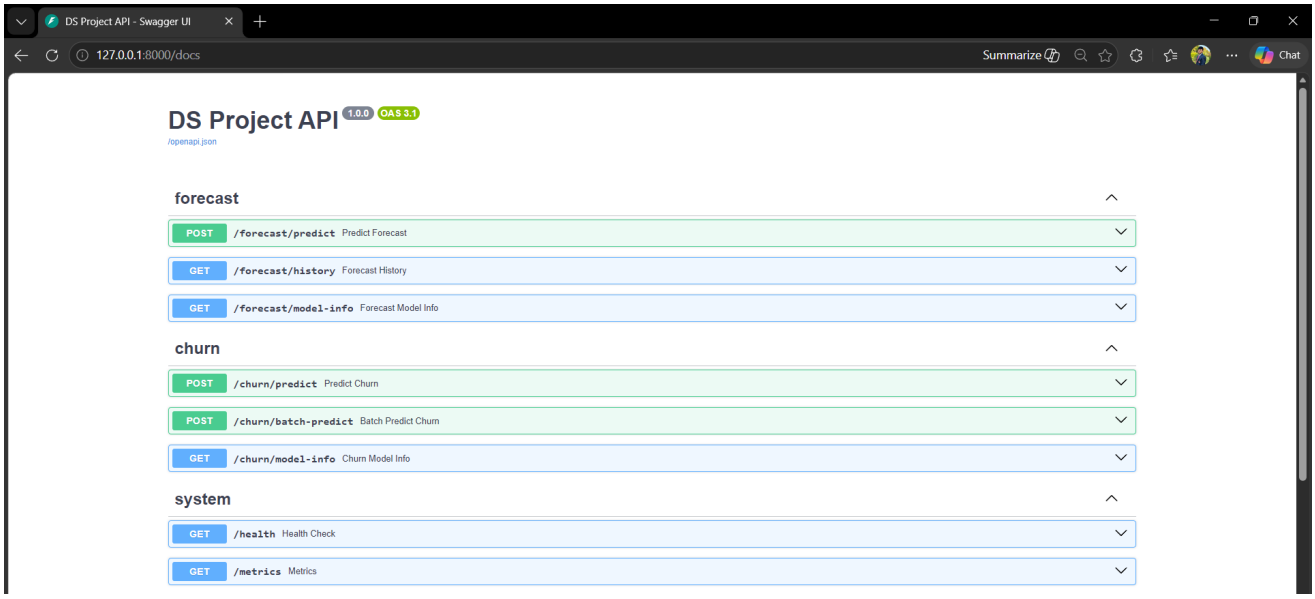

Figure: Swagger UI generated by FastAPI, showing the forecast, churn, and system endpoint groups used for inference and monitoring. File name: docs.png.

Health Endpoint

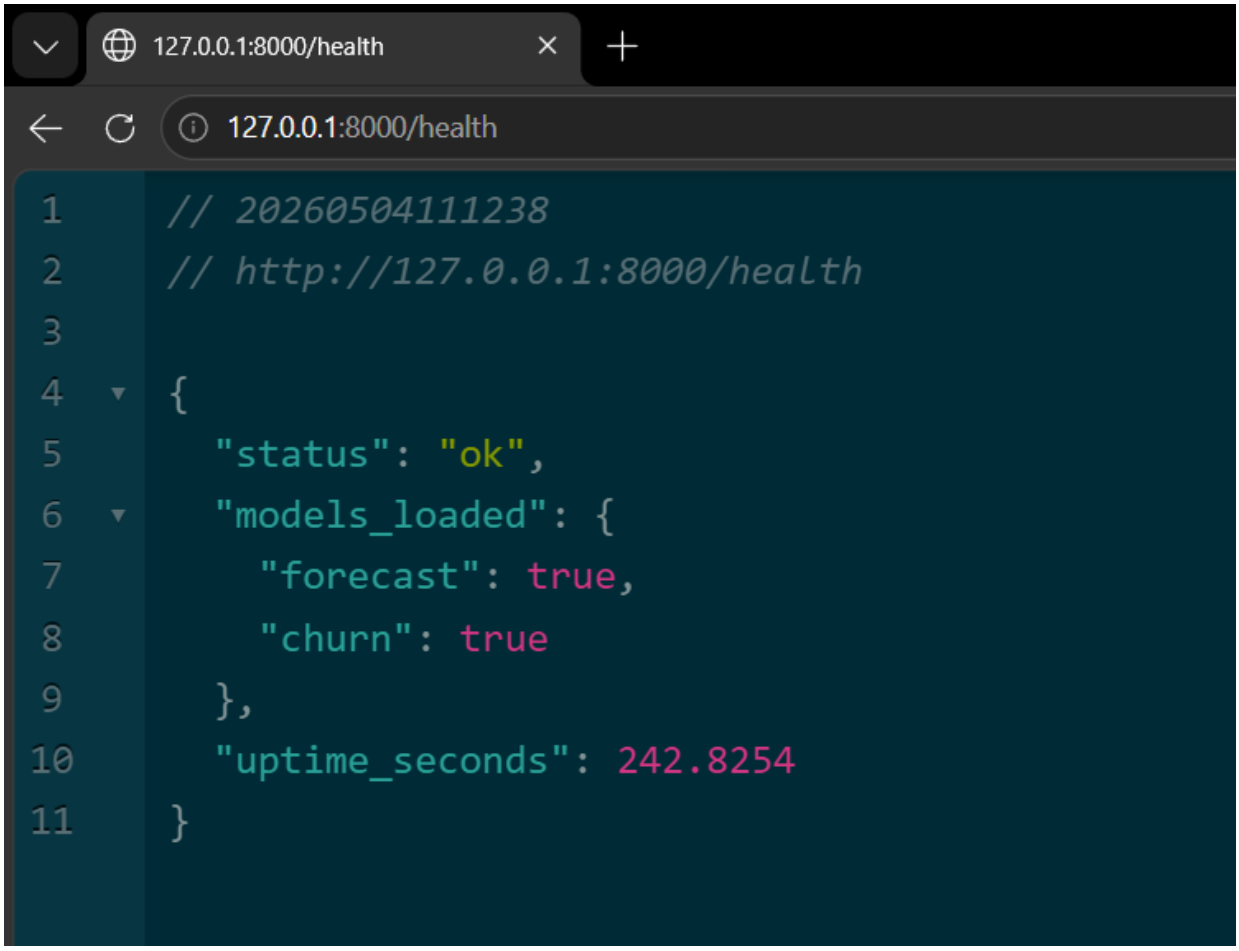

Figure: Runtime health response confirming the service is online and both forecasting and churn models were loaded successfully. File name: health.png.

Training Pipeline Output - Part 1

```
(.venv) PS C:\Users\priya\Desktop\Nexora> python scripts\train_all_models.py
2026-05-04 11:14:27.660455: I tensorflow/core/util/port.cc:113] oneDNN custom operations are on. You may see slightly different numeric
al results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF
_ENABLE_ONEDNN_OPTS=0`.
2026-05-04 11:14:30.009274: I tensorflow/core/util/port.cc:113] oneDNN custom operations are on. You may see slightly different numeric
al results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF
_ENABLE_ONEDNN_OPTS=0`.
2026-05-04 11:14:37,906 | INFO | src.data_processing.sales_preprocessor | Saved processed daily revenue to C:\Users\priya\Desktop\Nexor
a\data\processed\daily_revenue.csv
Forecast training summary
{
  "model_name": "ARIMA",
  "order": [
    5,
    0,
    0
  ],
  "stationary": true,
  "adf_p_value": 0.0,
  "train_points": 335,
  "test_points": 30,
  "last_train_date": "2023-12-01",
  "last_test_date": "2023-12-31",
  "metrics": {
    "MAE": 667.5171,
    "RMSE": 813.1325,
    "MAPE": 64.219
  }
}
2026-05-04 11:14:38.193151: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available C
PU instructions in performance-critical operations.
To enable the following instructions: AVX2 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
Churn training history summary
{
  "epochs_ran": 40,
  "best_val_loss": 0.1514,
  "final_train_loss": 0.1361,
  "final_val_auc": 0.9922
}
Churn evaluation metrics
{
```

Figure: Console output from the unified training script, including ARIMA training summary and persisted preprocessing artifacts. File name: model_01.png.

Training Pipeline Output - Part 2

```
Churn evaluation metrics
{
  "accuracy": 0.96,
  "precision": 0.7333,
  "recall": 1.0,
  "f1": 0.8462,
  "roc_auc": 0.9939
}
Classification report
              precision    recall  f1-score   support

           0       1.00      0.96      0.98        89
           1       0.73      1.00      0.85        11

    accuracy                           0.96       100
   macro avg       0.87      0.98      0.91       100
weighted avg       0.97      0.96      0.96       100

Training pipeline completed successfully.
Forecast metrics: {'MAE': 667.5171, 'RMSE': 813.1325, 'MAPE': 64.219} | Churn metrics: {'accuracy': 0.96, 'precision': 0.7333, 'recall'
: 1.0, 'f1': 0.8462, 'roc_auc': 0.9939}
(.venv) PS C:\Users\priya\Desktop\Nexora>
```

Figure: Console output showing churn evaluation metrics, the classification report, and successful pipeline completion. File name: model_02.png.

Interactive Dashboard

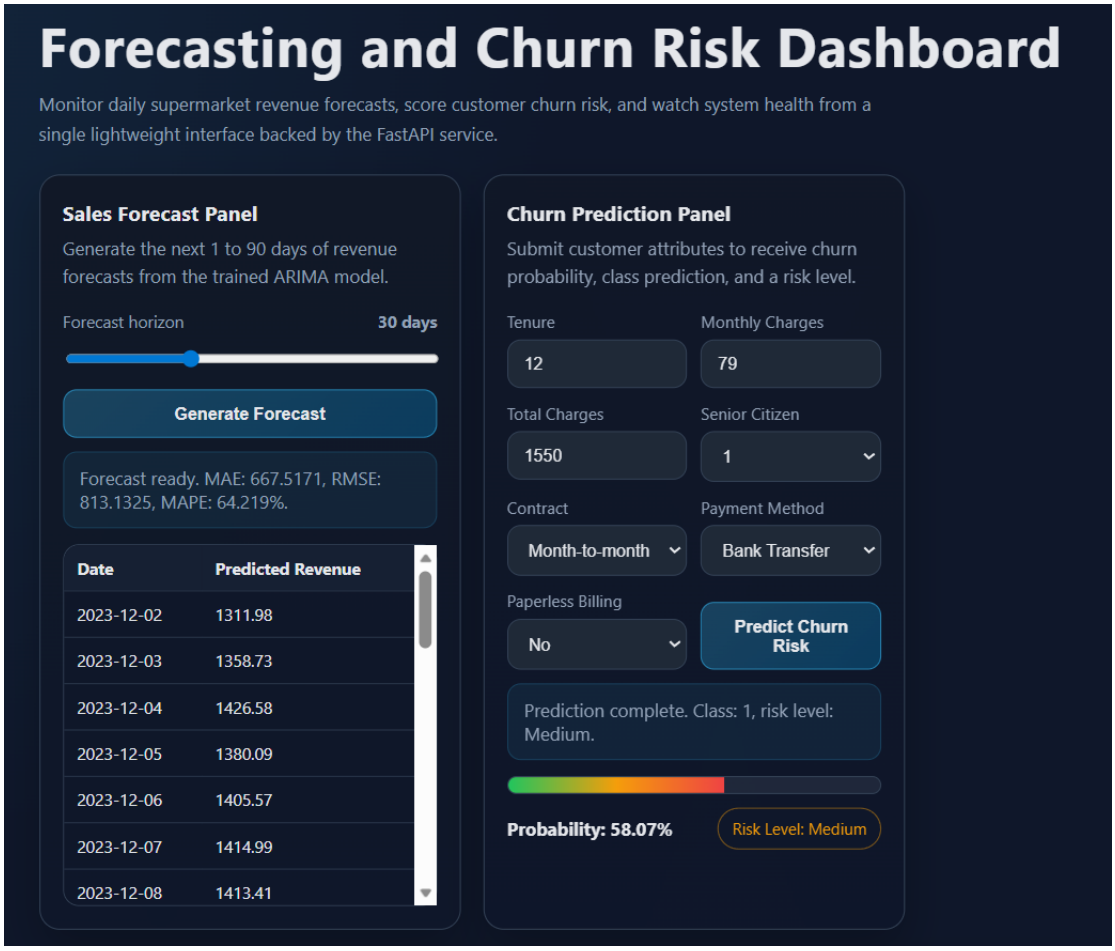

Figure: Frontend dashboard for sales forecasting and churn prediction, including forecast results, probability visualization, and risk labels. File name: results.png.

10. Limitations and Future Enhancements

The main limitation in the current system is the sales forecasting accuracy. Because the daily revenue series is volatile and no external regressors are included, ARIMA serves as a baseline rather than a fully optimized planning tool. Future work should evaluate richer seasonal models, additional lag features, holiday effects, product-line segmentation, and potentially Prophet or neural forecasting approaches if the business case requires better medium-range accuracy.

The churn model performs well on the available dataset, but the training sample remains small. Additional rows, longitudinal features, and customer interaction variables would likely improve generalization and make threshold tuning more meaningful. On the platform side, likely next steps include authentication, structured logging to an observability backend, CI-based report generation, and container validation on a Docker-enabled host.

11. Conclusion

Nexora demonstrates an end-to-end data science workflow that goes beyond notebook-only experimentation. It packages preprocessing, model training, inference, evaluation, API serving, monitoring, and a lightweight web interface into one coherent repository. The churn pipeline is already a strong showcase of supervised tabular learning, while the forecasting pipeline provides a clear baseline that can be improved iteratively. Together, these components make the project suitable as an academic report artifact, a portfolio project, or a foundation for future production hardening.